CHAPTER 22

Ad Click Event Aggregation

Aggregate billions of ad click events into accurate, near real-time counts for billing and reporting.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a system that **collects ad click events and aggregates them** (Facebook / Google Ads). Advertisers query clicks per ad over the last M minutes and top-N most clicked ads. The numbers drive **billing**, so correctness, idempotency and recoverability matter as much as speed.

Why interviewers ask it: Stream processing deep dive: exactly-once, windowing, watermarking, lambda vs kappa, hot spots, reconciliation.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Input data?	Log files from servers: ad_id, click_timestamp, user_id, ip, country.
Volume?	1 B clicks/day, 2 M ads; 30% YoY growth.
Queries?	Count of clicks for ad_id in last M minutes; top 100 most clicked ads per minute; filter by ip/country/user_id.
Edge cases?	Late events, duplicates, partial system failures.
Latency?	E2E a few minutes; aggregation queries ms.

3. Requirements

Functional

- Aggregate clicks per ad per minute.
- Return top-N ads in the last M minutes.
- Support filtering by attributes.

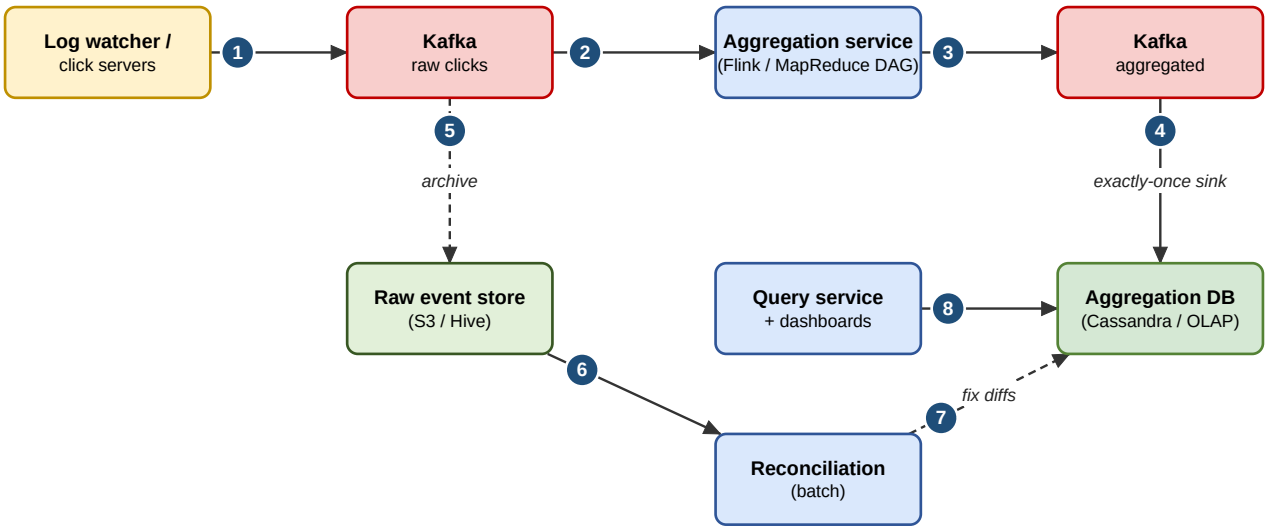
Non-functional

- **Correctness** (no loss, no double counting).
- Handle late and duplicate events.
- Resilient to failures and replayable.
- Latency in minutes end-to-end.

4. Back-of-the-envelope estimation

- 1 B/day = about **10,000 click QPS** average; peak 5x = **50,000 QPS**.
- Event about 0.1 KB -> 100 GB/day raw = about 3 TB/month. Aggregated data (ad_id, minute, count) is orders of magnitude smaller.

5. High-level design



Kappa-style stream path for speed, plus a batch reconciliation path on raw data for correctness.

Step	What happens
1	Click servers write events (ad_id, timestamp, user_id, ip, country) to logs; a watcher publishes them to the **raw clicks Kafka topic**, partitioned by ad_id.
2	The **aggregation service** (a map / aggregate / reduce DAG, e.g. Flink) consumes the stream. Map nodes filter and route by `ad_id`; aggregate nodes count per ad per minute.
3	Aggregates (per-minute counts, top-N) are published to a second Kafka topic. Windows use **event time** with a **watermark** for late events.
4	A sink writes to the **aggregation DB** with **exactly-once** semantics (checkpointed offsets + transactional or idempotent writes).
5	In parallel, every raw event is archived to the **raw event store** (S3/Hive) as the source of truth.
6	A nightly **reconciliation** batch recomputes counts from raw data and compares them with the streaming results.
7	Any discrepancies (for example from very late events) are corrected in the aggregation DB.
8	Advertisers query `clicks for ad X in the last M minutes` and top-N ads, served from pre-aggregated data in milliseconds.

6. Deep dive

Why store raw data too

- Keep raw events (cheap object storage) for **recalculation**, backfill after bugs, auditing and reconciliation. Aggregated data answers queries fast; raw data is the source of truth.

Aggregation service as a DAG (MapReduce)

- **Map** nodes: read from Kafka, filter, normalise, partition by `ad_id % N` to route events to the right aggregator. **Aggregate** nodes: count per `(ad_id, minute)` in memory. **Reduce** nodes: merge into top-N (e.g. heap merge) - e.g. 3 aggregate nodes each produce local top 3 -> reduce to global top 3.
- Pre-aggregate in the stream layer instead of per-query scans.

Time windows

- **Tumbling window** (fixed non-overlapping, e.g. per minute) for per-minute counts; **sliding window** (overlapping) for 'last M minutes' queries or top-N in last M minutes.
- **Event time vs processing time:** event time (click occurred) is accurate but late events happen; processing time is simple but less accurate. Use event time + **watermark** (extra wait window, e.g. 15 s) to include late arrivals; larger watermark = more accuracy but more latency; very late events are corrected by the batch reconciliation.

Delivery guarantees: exactly-once

- Billing needs exactly-once semantics. Use at-least-once delivery from Kafka plus **idempotent/transactional processing** and dedupe: store the offset and the aggregated result in one atomic step, or use distributed transactions/checkpointing (Flink two-phase commit sink).
- Duplicates come from client retries or consumer failure before offset commit; use a unique click id and **dedupe** window; on aggregator failure resume from the last checkpoint offset rather than the 'last read' offset.

Scaling

- Kafka: partition by ad_id; scale brokers/partitions; producers use batches; topics per ad_id range. Consumers scale by group rebalance - minimise rebalance latency with ready-made capacity.
- **Hot ad (hotspot):** many events for one ad overload one aggregator -> allocate extra aggregator nodes for that key (split key with salt then merge) via a resource manager.
- Cassandra scale by adding nodes (consistent hashing) for the aggregation DB.

Fault tolerance

- Aggregation nodes keep in-memory state -> checkpoints and Kafka offsets enable recovery by replay (snapshot state + resume offset); restart from the last checkpoint.

Data monitoring and reconciliation

- Track latency (timestamps along the pipeline), Kafka consumer lag, system resources. **Reconciliation:** nightly batch job sorts raw events per ad per minute, recomputes counts and compares with the streaming results; fix discrepancies. Late events are accounted for here.

Alternative architecture

- **Lambda** (separate stream + batch code paths; two codebases) vs **Kappa** (single streaming path, replay from Kafka for reprocessing). Kappa is preferred for simplicity.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Architecture	Lambda / Kappa	Kappa: one code path; replay raw log for corrections.
Window time	Event time / processing time	Event time + watermark for accuracy.
Watermark	Short: low latency / long: more accurate	Short + batch reconciliation to fix stragglers.
Sink	At-least-once + dedupe / exactly-once transactions	Exactly-once for billing.

8. Scaling, reliability, failure modes

- Partition by ad_id everywhere; handle hot keys with salting.
- Store 1-minute aggregates, roll up to hours/days for older queries.
- Archive raw data to cold storage after reconciliation window.

9. Likely follow-up questions

- How do you ensure no double billing on duplicate events?
- How do you handle clicks that arrive hours late?
- How would you detect click fraud?
- How do you handle one ad suddenly receiving 100x traffic?

10. Common mistakes

- Skipping raw data retention (no recomputation).
- Confusing event time with processing time.
- Ignoring hot partitions.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"The input is click log files with ad ID, timestamp, user, IP and country. Volume is about 1 B clicks per day, 10,000 QPS average and 50,000 at peak. The queries are click counts per ad for the last M minutes, and the top 100 ads per minute. Since this drives billing, correctness, deduplication and late-event handling matter more than raw speed."
2. High-level design	10 min	"I'd use a streaming pipeline: logs into Kafka, a stream aggregation service, aggregated results to a second Kafka topic and then to an aggregation DB. In parallel, raw events go to cold storage, and a nightly reconciliation job recomputes from raw data to fix discrepancies."
3. Deep dive	25 min	"Aggregation is a map-aggregate-reduce DAG partitioned by ad ID. For windows I use tumbling windows for per-minute counts and sliding windows for 'last M minutes', based on event time with a watermark for late events. For exactly-once I checkpoint offsets with state and use idempotent or transactional sinks. Hot ads get extra aggregator nodes. I'd also compare lambda and kappa, and prefer kappa for a single code path."
4. Wrap up	4 min	"Monitoring: end-to-end latency, consumer lag, duplicate rate. Fault tolerance: restart from checkpoints. Beyond this: click-fraud detection and rollups for older data."

12. 60-second recap / practice checklist

- Log -> Kafka -> stream aggregation (map/aggregate/reduce, tumbling + sliding windows) -> Kafka -> aggregation DB; raw events to cold storage.
- Correctness: exactly-once, dedupe, watermark, checkpoints, nightly reconciliation.
- Scale by partitioning on ad_id; mitigate hot keys.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Flink: Timely stream processing (event time, watermarks)
• The Dataflow Model (Google)
• Kafka exactly-once semantics